SQL
S4

EDC- 2026
Sh. Dowlatshahi

Session 4 :Aggregates

- Use the aggregates: (COUNT, SUM, MIN/MAX, AVG)
- Use Group By, Having
- First assignment (startups)

Course content

S1: Introduction to Databases & SQL

S2: DDL

S3: DML

S4: Aggregation and Basic Analysis

S5: Working with Multiple Tables

S6: Advanced Queries

S7: Final presentation & Exam

Case studies: film & Country_data

Return

To retrieve the affected data during an INSERT, UPDATE, or DELETE operation, you can use the RETURNING clause.

Return- Example

Updates the rating for 'Inception' and returns the movie title and new rating:

```
UPDATE film

SET rating = 9.0

WHERE title = 'Inception'

RETURNING title, rating;
```

LIMIT

LIMIT Clause is used to restrict the number of rows returned by a query.

It's like saying: "Just give me the first x results."

```
SELECT * FROM film
LIMIT 5;
```

This shows only the first 5 rows from the film table.

LIMIT & ORDER BY

```
SELECT * FROM film
ORDER BY rating DESC
LIMIT 3;
```

This query returns the 3 films with the highest rating from the film table.

AS

AS in PostgreSQL is a keyword used for aliasing.

It is part of the SQL syntax that lets you assign an alias (a temporary name) to a column or table in your query.

Alias

- You can use column aliases in:

SELECT – to rename output columns

ORDER BY – because it comes after SELECT

Subqueries – to reference calculated columns

- You can't use aliases in WHERE / GROUP BY / HAVING

(Because those run before aliases are created).

EXAMPLE- AS

```
SELECT product_name, price * quantity AS total_sales
FROM sales
ORDER BY total_sales DESC;
```

– total_sales is defined in SELECT and used in ORDER BY.

1. Alias for a column:

You can rename a column in the result set.

SELECT rating AS film_rating FROM film;

Here, the column rating will appear as film_rating in the query result.

2. Alias for a table:

You can give a shorter or more meaningful name to a table, often useful in joins.

```
SELECT s.name, s.grade

FROM students AS s

WHERE s.grade > 90;
```

students AS s means you can use s as a shortcut for the students table in the query.

DISTINCT

DISTINCT is a keyword used in a SELECT statement to remove duplicate rows from the result set. It ensures that the rows returned are unique based on the selected columns.

SELECT DISTINCT column_name FROM table_name;

This will return only unique values of column_name => any duplicates will be removed.

IF EXISTS/NOT EXISTS

Use IF NOT EXISTS or IF EXISTS clauses to avoid errors when creating or deleting tables.

(These clauses prevent your script from failing due to existence-related errors, especially helpful when running automation, migrations, or repeated queries.)

```
-- If a table with this name already exists, the operation will not be executed.

CREATE TABLE IF NOT EXISTS test (
        column1         INTEGER,
        column2         VARCHAR(255)
        );



-- Dropping the test table (if it exists).
DROP TABLE IF EXISTS test;
```

Using Constraints to Enforce Data Rules in SQL Tables

A CONSTRAINT in databases is a rule applied to a table's columns to ensure the integrity, accuracy, and reliability of the data.

Constraints restrict the kind of data that can be inserted, updated, or deleted in the table.

Common types of constraints:

- PRIMARY KEY: Uniquely identifies each row in a table.
- FOREIGN KEY: Ensures the value in one table matches a value in another table (maintains relationships).
- UNIQUE: Ensures all values in a column are different.
- NOT NULL: Ensures a column cannot have NULL (empty) values.
- CHECK: Ensures values in a column meet a specific condition (e.g., price > 0).

- During create and Alter

What is a PRIMARY KEY?

A PRIMARY KEY is a column or group of columns that uniquely identifies each row in a table.

Values in the primary key must be:

- Unique (no duplicates allowed)
- Not NULL (every row must have a value)

PRIMARY KEY:

Formula:
column_name data_type PRIMARY KEY

Example:
id SERIAL PRIMARY KEY

–id is the unique identifier for each row.

What is a FOREIGN KEY?

A foreign key makes sure that values in one table match values in another table’s primary key.

It helps keep data correct by stopping wrong or unrelated values from being added.

FOREIGN KEY:

Formula:
column_name data_type REFERENCES other_table(column_name)

Example:

city_id INT REFERENCES city(id)

We use a foreign key to link two tables together. This means the city_id links to the id in the city table (other table).

UNIQUE:

Formula:
column_name data_type UNIQUE

Example:
email VARCHAR(100) UNIQUE

Ensures no two rows have the same email.

CREATE, ALTER

NOT NULL:

Formula:

column_name data_type NOT NULL

Example:
username VARCHAR(50) NOT NULL

A value must always be provided for username.

Does not allow null values during CREATE and ALTER

CHECK:

CHECK

column_name data_type CHECK (condition)

Example:
age INT CHECK (age >= 18)

Only allows age values that are 18 or above.

DEFAULT:

Formula:
column_name data_type DEFAULT default_value

Example:
created_date DATE DEFAULT CURRENT_DATE

If no value is given, it will automatically be set to the current date.

Analyze Data

1. Row Count

Indicates how many records matched your query.

```
SELECT COUNT(*)
FROM employees
WHERE department = 'HR';
```

2. Column Values

Review returned values.

```
SELECT name, hire_date
FROM employees
WHERE hire_date > '2023-01-01';
```

Check for:

- Unexpected NULLs:

```
SELECT *
FROM employees
WHERE email IS NULL;
```

- Outliers (e.g., a salary of 1,000,000 when the average is 60,000):

```
SELECT name, salary
FROM employees
WHERE salary > 100000;
```

- Typos

3. Ordering

Use ORDER BY to sort the result rows by one or more columns.

```
SELECT name, salary

FROM employees

ORDER BY salary DESC;
```

This query retrieves the name and salary of all employees and sorts them from highest to lowest salary.

By default, it sorts in ascending order (from smallest to largest).

Find the Max/MIN Value with ORDER BY & SELECT

– Max:

```
SELECT column_name
FROM table_name
ORDER BY column_name DESC
LIMIT 1;
```

```
SELECT MAX(column) FROM TABLE;
```

–Min:

```
SELECT column_name
FROM table_name
ORDER BY column_name ASC
LIMIT 1;
```

4. Practical Tips

- Cross-check totals using COUNT(), SUM(), AVG() for consistency.

- Use filters like LIMIT to preview large datasets.

Summarizing data

When working with data, it's important not only to examine individual records but also to get a **big-picture** view of the entire dataset.

SQL provides powerful aggregate functions that help summarize data by performing calculations across multiple rows and **returning a single value.**

COUNT() – counts the number of rows

AVG() – calculates the average

SUM() – adds up values

MIN() – finds the smallest value

MAX() – finds the largest value

These functions are used in a SELECT statement to return a single summarized result.

COUNT() – Number of Values

Counts how many rows match a condition.

Example:

SELECT COUNT(*) FROM sales;

– Counts the total number of sales records.

Example:

SELECT COUNT(score) FROM students;

– COUNT(column) counts only **rows** where that column has a value (not NULL).

SUM() – Total Sum

Adds up the values in a numeric column.

Example:
SELECT SUM(amount) FROM sales;

Calculates the total amount of all sales.

MIN() / MAX() – Smallest and Largest

Finds the smallest or largest value in a column.

Example:

SELECT MIN(price), MAX(price) FROM products;

Shows the lowest and highest product prices.

AVG() – Average Value

Finds the average of a numeric column.

Example:

SELECT AVG(rating) FROM reviews;

Gives the average rating from the reviews table.

Use **GROUP BY** to group rows that have the same values in specified columns.

Required if you want to apply aggregate functions like SUM(), COUNT(), AVG(), etc., on groups of **rows.**



GROUP BY

Example:

SELECT department, SUM(salary) FROM employees
GROUP BY department;

This query shows the total salary paid for each department in the company.

HAVING

HAVING is used to filter the results after rows have been grouped using GROUP BY.

HAVING applies conditions to the grouped data, not individual rows.

You can use aggregate functions like SUM(), AVG(), or COUNT() in the HAVING clause to filter groups that meet certain conditions.

Can't use column aliases in HAVING.

Example:

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 100000;
```

This query finds departments that pay more than 100,000 in total salaries and shows the department name with the total salary amount.

```
SELECT department, AVG(salary), MAX(salary)
FROM salary_infos
WHERE employment_date >= '2023-01-01'
GROUP BY department;
```

This query shows, for each department, the average and highest salaries of employees hired since January 1, 2023.

```
SELECT department, AVG(salary), MAX(salary)
FROM salary_infos
WHERE employment_date >= '2023-01-01'
GROUP BY department
HAVING COUNT(*) > 1;
```

This query shows the average and highest salaries for departments that have at least two employees hired since January 1, 2023.

If the number of employees is less than two, it won't be shown in the result set.

HAVING is like a WHERE, but it's used **after** grouping data with GROUP BY.

- WHERE filters rows before grouping.
- HAVING filters groups after grouping.

Recap

- Use aggregate functions like MAX(), COUNT(), etc. to summarize data.
- Use GROUP BY to **group rows** before aggregation.
- Use WHERE to **filter rows** before aggregation.
- Use HAVING to **filter groups** after aggregation.